Control Path

Boot:

When the SoC is powered up, the reset controller resets memory, bus, arbiter, CPU and peripherals.

CPU runs code from internal BOOT ROM:

1. Stack pointer points to first instruction in Boot ROM
2. CPU configures SPI and makes it ready for transfer
3. CPU pulls startup code from **external flash** to SRAM (or ITCM?)

Startup code:

- CPU sets registers of debug module, arbiter, interrupt controller
- Start watchdog
- Direction of each GPIO register needs to be set, GPIO interrupts need to be enabled, debounce needs to be enabled for some registers
- I2S settings configured
- DMA configured
  DMA channel 0: I2S RX buffer to SRAM
  DMA channel 1: SRAM to FFT Scratchpad
  DMA channel 3: IFFT Scratchpad to SRAM
  DMA channel 4: SRAM to I2S TX buffer
  (We are considering that the CPU will move data from FFT Scratchpad to DTCM for filtration, and DTCM to IFFT after filtration. Compare this to the scenario of using SRAM instead of or along with DTCM.)
- CPU loads filtration instructions from **flash** into I-TCM through SPI
- CPU loads trigger vectors from **flash** into D-TCM through SPI
- CPU loads twiddle factors from **flash** into FFT scratchpad, and inverse twiddle factors from flash into IFFT scratchpad, through SPI

Realtime:

1. PCM data from MIC stored in I2S RX FIFO buffer (size can be 512 samples? - needs to be large enough to hold samples until DMA reads them.)
2. Once 512 samples are collected in the FIFO buffer, I2S's ready_in raises an interrupt to the CPU. CPU configures DMA channel 0 to move data from I2S FIFO to SRAM.
3. DMA channel 0 moves data from I2S RX FIFO buffer to SRAM.
4. Once 512 samples have been moved to SRAM, DMA channel 0 raises an interrupt to CPU, which requests DMA channel 1 to transfer 512 samples from SRAM to the FFT scratchpad.
5. After 512 samples are transferred to FFT scratchpad, DMA channel 1 raises an interrupt to the CPU. CPU sets fft_start and FFT operations happen. The results of FFT operations are overwritten in the scratchpad.
6. Once the FFT operations are done, the fft_done flag is set, which raises an interrupt to the CPU. CPU transfers the 512 resultant samples from FFT Scratchpad to DTCM.
7. Once the transfer is complete,
    a. DMA channel 1 transfers the next 512 samples from SRAM to FFT Scratchpad
    b. CPU starts the filtration process for the current 512 samples
8. If the filtration process by the CPU is interrupted (to configure DMA etc), the ISR runs, and then filtration is continued. (figure out exact ISRs and how to store necessary data until filtration is resumed)
9. Once filtration is completed, the CPU transfers data from DTCM to IFFT Scratchpad.
10. After 512 samples are transferred to IFFT scratchpad, CPU sets ifft_start and IFFT operations happen. The results of IFFT operations are overwritten in its scratchpad.
11. Once the IFFT operations are done, the ifft_done flag is set, which raises an interrupt to the CPU. CPU requests DMA channel 3 to transfer the 512 resultant samples from IFFT Scratchpad to SRAM.
12. Once 16 samples are transferred, DMA channel 3 sets a flag IFFT_to_SRAM_started. When (IFFT_to_SRAM_started & ready_out) is true, an interrupt is sent to the CPU, which configures DMA channel 4 to move data from SRAM to I2S TX FIFO buffer.

13. The samples are sent from the I2S TX FIFO buffer to the external DAC and headphones.

Stuff to think about:

- How many frames do we need to store at a time, how should CPU keep track of the base address of each frame

- FFT Block -> SRAM to DTCM or FFT Block -> DTCM directly?
- Can we use DMA to transfer data into the DTCM by adding another port?
- To start the next process, can we just use flags from DMA, FFT block etc or better to send interrupt to CPU to start?
- Will each DMA channel be a separate master for the arbiter and have a separate line to the interrupt controller?